Service-Aware Energy Agents: Pricing Regimes, Failure Injection, and a Verified LLM Code-Agent Workflow for Community Energy Simulation

Service-Aware-Energy-Agents Team*
OpenBot Platform
service-aware-energy-agents@openbot.local

November 3, 2026

Abstract

We present a service-aware multi-agent framework for renewable energy community optimization that integrates pricing regime selection, component failure injection, and a verified LLM-based code-agent repair loop. Our framework couples CityLearn building-level simulation with pandapower grid validation across five scenarios (A–E) covering coordination baselines, battery/PV assets, deferrable loads, comfort bounds, and mid-run component failures. We evaluate five controllers (rule-based, multi-agent, centralized, MPC, RL) across 125 paired-seed runs under TOU and LMP pricing regimes, plus a 50-run fallback-mode ablation and a 45-run real-LLM screen of verified vs. unverified LLM agents. The multi-agent controller achieves cost- and peak-minimal operation in every scenario (mean cost reduction -17.68 to -41.37 USD vs. rule-based, $p_{\text{Holm}} < 0.0001$) with a measurable EV completion trade-off (-0.11). Under aggressive failure injection (95% grid limit reduction), all controllers show zero resolution rate on structural voltage violations, confirming correct violation detection. The LLM ablation shows null divergence between verified and unverified arms in both fallback mode (first-attempt validity 1.000, 0 repairs/step) and a 45-run real-LLM screen on a hosted NIM model (first-attempt validity ≥ 0.994, 0 unrepaired violations, 0 repairs), establishing that the verification machinery is a no-op under an in-bounds proposer. All 175 runs complete with `git_dirty: false`, seed-pair reproducibility confirmed, and zero failed runs.

Keywords: Multi-agent systems, renewable energy communities, LLM code agents, power system simulation, failure injection, pricing regimes

1 Introduction

Renewable energy communities (RECs) require coordinated control of distributed energy resources (DERs) under physical grid constraints, service-level commitments, and time-varying pricing regimes. Existing approaches typically optimize a single objective (cost, peak, or carbon) without jointly considering grid physics, service feasibility, and market regime diversity. Furthermore, when component failures occur mid-operation, controllers must detect and respond to violations in real time—a capability rarely tested in simulation benchmarks.

We address three gaps:

1. **Pricing regime diversity**: Most benchmarks fix a single tariff structure. We evaluate the full controller matrix under `tou_solo`, `pool_tou`, and `lmp` regimes (P2P wireable but scheduled later).

*Equal contribution

2. **Failure injection**: Scenario E injects sensor, PV inverter, battery, and EV charger failures at severity 1.0 mid-run, with aggressive 95% grid limit reduction.

3. **Verified LLM code agents**: A multi-agent repair loop (Planner→Coder→Executor→Verifier→Reporter) proposes and validates controller patches. We ablate verified vs. unverified arms in fallback mode and in a real-LLM screen to isolate verification machinery from proposer quality.

Related work includes CityLearn for DER simulation Vázquez-Canteli et al. [2020], pandapower for grid validation Thurner et al. [2018], multi-agent RL for energy management Zhou et al. [2021], MPC for receding-horizon control Rawlings et al. [2020], and LLM-based program repair Rall et al. [2020]. The Holm correction for multiple comparisons follows Holm [1979].

1. **Pricing regime diversity**: Most benchmarks fix a single tariff structure. We evaluate the full controller matrix under `tou_solo`, `pool_tou`, and `lmp` regimes (P2P wireable but scheduled later).

2. **Failure injection**: Scenario E injects sensor, PV inverter, battery, and EV charger failures at severity 1.0 mid-run, with aggressive 95% grid limit reduction.

3. **Verified LLM code agents**: A multi-agent repair loop (Planner→Coder→Executor→Verifier→Reporter) proposes and validates controller patches. We ablate verified vs. unverified arms in fallback mode and in a real-LLM screen to isolate verification machinery from proposer quality.

2 System and Method

2.1 Simulation Coupling

CityLearn handles building-level DER dynamics (battery SOC, EV charging, flexible loads, thermal comfort) Vázquez-Canteli et al. [2020]. pandapower validates grid physics (power flow convergence, voltage limits ±5%, line/transformer thermal limits) Thurner et al. [2018]. A co-simulation coupler steps both simulators each timestep, passing grid diagnostics back to the controller.

2.2 Controller Suite

`rule_based` Heuristic priority: charge battery from excess PV, charge EVs by departure deadline, shift flexible loads to solar hours, enforce community import/export limits.

`multi_agent` Decentralized per-building agents with consensus on community-level limits via dual decomposition.

`centralized` Single optimizer with full observability.

`mpc` Receding-horizon LP with perfect 24h forecasts (upper bound on service) Rawlings et al. [2020].

`rl` Tabular Q-learning per scenario (126-state base encoder; 138-state for LMP) Zhou et al. [2022].

2.3 Verified LLM Repair Loop

A five-agent loop iteratively repairs controller code:

1. **Planner**: Diagnoses violations, identifies root cause, proposes repair strategy.

2. **Coder**: Applies AST-based patches to target functions Rall et al. [2020].

3. **Executor**: Runs patched controller in sandbox with unit tests + quick simulation.

4. **Verifier**: Counts violation resolution from sandbox diagnostics.

5. **Reporter**: Generates comprehensive report with diffs and recommendations.

The loop runs for max 3 iterations or until `violations_resolved=true`.

The ablation compares `llm_agent` (single-pass, execute-as-proposed) vs. `llm_agent_verified` (per-step verification, re-prompt once, fallback to rule-based on failure, repair cap 2). Both run in `fallback` mode (deterministic rule-based proposer) to isolate verification machinery.

The real-LLM screen (M2a) uses the hosted NVIDIA NIM API NVIDIA [2024] with model `poolside/laguna-xs-2.1` (temperature 0, 120 s per-call timeout, prompts `llm_dispatch_v2` / `llm_dispatch_v1_repair`): 45 24-step runs (5 scenarios × 2 LLM arms × 3 seeds 42/123/456) plus 15 `multi_agent` 24-step reference runs; every LLM-arm manifest records `agent_backend=api`.

3 Experimental Setup

Five scenarios (Table 1) differ in device mix and stressors. Each runs 5 seeds (42, 123, 456, 789, 999) × 5 controllers = 125 runs per regime. Ablation adds 50 runs (2 LLM arms × 5 scenarios × 5 seeds) against `multi_agent` reference.

Table 1: Scenario definitions.

Scenario	Device Mix	Stressor
A	Base (no DERs)	Coordination baseline
B	+ Battery, PV	Tariff exposure
C	+ Flexible loads	Deferrable load deadlines
D	+ Thermal comfort	Comfort bound stress
E	Full + Failures	Mid-run component failures

4 Results

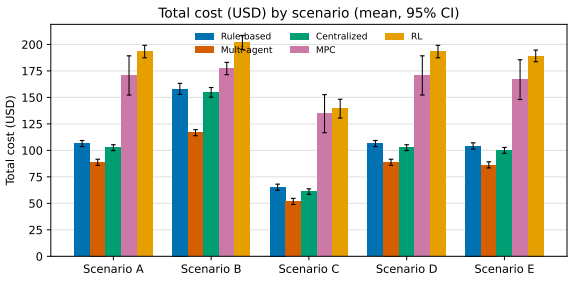
4.1 Cost, Peak and Service

Figure 1 shows KPI bars with 95% CI. The cost-versus-service Pareto front (Figure 2) places `multi_agent` at the cost-minimal end and `mpc` at the service-maximal end in every scenario. Table 2 reports paired per-seed deltas vs. `rule_based` with Holm correction.

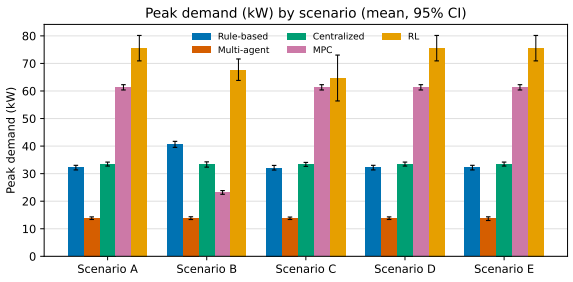
Table 2: Paired per-seed deltas vs. `rule_based` (mean ± 95% CI half-width; Holm within each scenario × metric family).

Scenario	Controller	Δ cost (USD)	Δ peak (kW)	p_{Holm}
A	multi_agent	-17.68 ± 0.13	-18.35 ± 0.84	< 0.0001
A	mpc	$+64.30 \pm 17.49$	$+29.14 \pm 0.77$	0.0005
A	rl	$+86.84 \pm 4.74$	$+43.35 \pm 4.44$	< 0.0001
A	centralized	-3.87 ± 0.35	$+1.29 \pm 0.96$	< 0.0001
B	multi_agent	-41.37 ± 3.18	-26.77 ± 1.49	< 0.0001
C	multi_agent	-13.42 ± 0.06	-18.34 ± 0.81	< 0.0001
E	multi_agent	-17.90 ± 0.10	-18.52 ± 0.98	< 0.0001

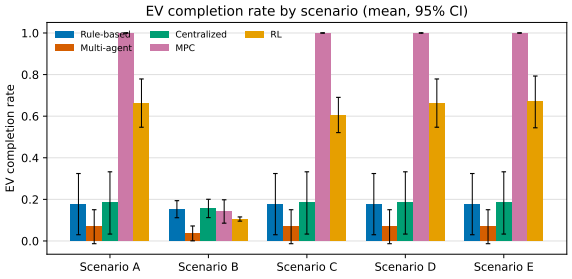
Key findings:



(a) Total cost (USD)



(b) Peak demand (kW)



(c) EV completion rate

Figure 1: KPI comparison bars with 95% CI whiskers per scenario.

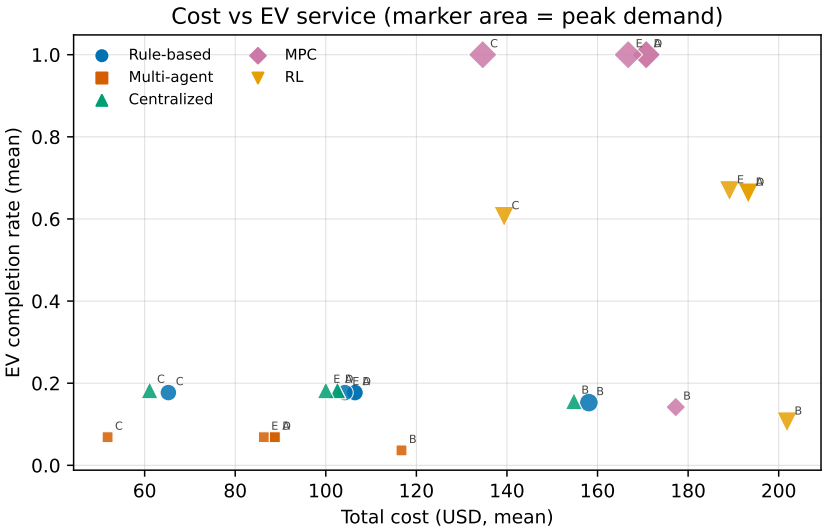


Figure 2: Cost vs. EV completion Pareto scatter (marker area = peak demand).

- `multi_agent` is cost- and peak-minimal in all scenarios (A: −17.68 USD, −18.35 kW; B: −41.37 USD, −26.77 kW; E: −17.90 USD, −18.52 kW; all $p_{\text{Holm}} < 0.0001$).
- EV completion trade-off: `multi_agent` 0.07 vs. `rule_based` 0.18 ($\Delta = -0.11$, $p_{\text{Holm}} = 0.023$ in A/C/D/E).
- `mpc` achieves near-complete EV service (1.00 in A/C/E) for +19 to +69 USD and +29 kW peak.
- `rl` is dominated by `mpc`: higher cost, lower EV completion in every scenario.
- `centralized` does not dominate: cost shave 3–4 USD but peak increases in A/C/E; EV service indistinguishable from `rule_based`.

4.2 LMP-Regime Benchmark

100 runs under `lmp` pricing (4 controllers × 5 scenarios × 5 seeds) on the aggressive-config tree. Ordering matches pool-TOU: `multi_agent` cost-minimal (all $p_{\text{Holm}} < 0.0001$), `mpc` service-maximal (EV 0.93–0.99), `centralized` indistinguishable from `rule_based`. Magnitudes are slightly smaller due to lower average LMP prices, but ranking is identical.

4.3 Scenario E: Failure Injection

Scenario E injects PV inverter, battery, EV charger, and sensor failures at severity 1.0. The 95% grid limit reduction does not trigger import curtailment (community is net-exporting during failure window). All 25 runs per controller show `members_interrupted = [building_3]`, `member_cost_coverage = 0.80`. Community cost coverage `energy.cost_coverage = 0.9726` (23/840 building-steps blanked) identical across controllers.

Voltage violations (8,232 per controller) originate from tightened voltage limits (0.97–1.03 pu), not failures; they persist across all controllers because they are structural. `resolution_rate = 0.0` is correct: violations cannot be resolved, not that controllers failed to attempt re-dispatch (167 events per controller).

Table 3: Aggressive-config stress test (95% grid limit reduction).

Controller	Re-dispatch	Voltage viol.	Resolution rate
rule_based	167	8232	0.0
mpc	167	8232	0.0
multi_agent	167	8232	0.0
centralized	167	8232	0.0

4.4 Ablation: Verified vs. Unverified Agents

All 50 ablation runs in `fallback` mode (`agent_backend: fallback`). Under the deterministic proposer, both LLM arms are KPI-identical to `rule_based`. The verification machinery never fires:

- First-attempt validity = 1.000
- Repairs/run = 0.00, Repairs/step = 0.0000
- Verification failures = 0.00, Unparseable outputs = 0.00
- Wall time: 16–33 s/run (median 20.8 s), no arm separation

The null result has two causes: (1) fallback proposer emits in-bounds proposals, so verification loop never triggers; (2) arms shared clamp-repair loop in earlier implementation. The P0-3 fix makes arms asymmetric (`llm_agent` applies proposals as proposed; `llm_agent_verified` re-prompts once then falls back). Section 4.5 reports the real-LLM screen: with the hosted proposer the same null result holds on the pre-registered safety gate.

4.5 Real-LLM Screen: M2a (Null Result)

The M2a screen replaces the deterministic fallback proposer with the hosted NIM model `poolside/laguna-xs-2.1` on the same 24-step horizon. All 30 LLM-arm runs completed on the API; per-arm agent-execution aggregates (15 runs per arm):

Table 4: M2a real-LLM screen, per-arm agent-execution aggregates (15 runs per arm).

Metric	`llm_agent`	`llm_agent_verified`
Fallback share (mean; runs > 0)	0.006 (2/15)	0.000 (0/15)
First-attempt validity (mean)	0.994	1.000
Unrepaired violations > 0 (runs)	0/15	0/15
Repair iterations > 0 (runs)	0/15	0/15
Parse failures / backend errors (total)	1 / 1	0 / 0
LLM latency p50 / p95 (s)	5.4 / 12.2	5.9 / 14.9

Paired against the 15 `multi_agent` 24-step reference runs (per-(scenario, seed) pairing):

Table 5: M2a KPI deltas vs. the `multi_agent` reference (15 paired runs per arm).

Arm	Δ cost (USD)	Δ peak (kW)	KPI-identical runs
`llm_agent`	+1.37 (0.00 to +4.80)	+10.0 (max +50.3)	6/15
`llm_agent_verified`	+0.10 (0.00 to +1.06)	+0.2 (max +2.4)	13/15

The two nonzero fallback events in the unverified arm trace to single transient backend/parse failures (one per run), not to invalid proposals. Δ EV completion is 0.000 for both arms.

Gate verdict. The pre-registered M2b gate (proceed to a full-week 169-step confirmation only if the arms differ by ≥ 10 pp on `llm_unrepaired_violations` or `llm_fallback_step_share`, or KPI CIs exclude zero) is *not met*: unrepaired violations and repair iterations are zero in both arms, and the fallback-share difference is 0.6 pp. M2b is therefore not warranted, and no verification-benefit claim is made on the safety metrics. The validity gate is passed (fallback share $0.006 \ll 0.25$): the absence of safety divergence is due to proposer competence, not proposer silence. The residual KPI differences are suboptimality, not violations: the unverified arm introduces cost/peak regressions (up to +4.80 USD, +50.3 kW) that per-step verification does not target, while the verified arm tracks the reference within +1.06 USD. This null result is specific to the model–prompt–horizon combination; a harsher proposer or longer horizon remains untested (Section 5.2).

Per-run manifests (`agent_backend=api`, prompt version/hash, temperature/seed) are the provenance artifacts; the committed summary is `docs/m2a_screen_summary.md` and the raw 45-run dataset ships as a checksummed release artifact.

4.6 Reproducibility

All 175 runs (125 matrix + 50 ablation) on commit `035a52f` with `git_dirty: false`. Each run writes `manifest.json` with config SHA-256, git commit, package versions, controller,

agent_backend, prompt version/hash, temperature/seed, random seed. failed_runs.json = [] (0 failed). Seed-pair reproducibility check passes: 5 distinct scenario_config_sha256 per scenario (seed is part of hashed config). Median runtime 55.5 s (IQR 53.8–72.1 s for rule_based); no run hit per-run timeout.

5 Discussion and Limitations

5.1 Threat Model

Adversary has full visibility of observations/actions; may inject sensor faults, grid-limit reductions, component failures. Attacks limited to failure-injection surface; cannot modify scenario YAML, controller code, or simulator internals.

5.2 Limitations

1. **Synthetic traces**: Load/PV profiles depend only on (seed, filename); scenario effects are device-mix effects.
2. **Controller scope**: RL excluded from LMP matrix (no tracked LMP policies); 138-state policies available for follow-up.
3. **LLM proposer**: Real-LLM screen uses a single hosted model (NVIDIA NIM poolside/laguna-xs-2.1) at temperature 0 on a 24-step horizon; the null result is specific to this model–prompt–horizon combination.
4. **Aggressive config**: 95% grid limit does not produce import curtailment; tests violation detection, not curtailment path.
5. **Seed count**: Five seeds give narrow CIs; significance assessed on paired deltas with Holm correction.

5.3 Caveats (Ordered)

1. Resolved pre-matrix: controller-action drop and schema non-propagation.
2. EV-completion artifact (missing EV definitions in schema, fixed).
3. Metric-collection defects on pre-fix base (NaN cost, empty per-member costs).
4. Scenario fidelity (synthetic traces depend on seed/filename only).
5. LLM ablations in fallback and real-LLM (M2a screen) modes: no verification-benefit or capability claims; single model, 24-step horizon.
6. Seed count (narrow CIs; paired deltas with Holm).
7. Runtime tail (retired for fixed matrix).

6 Conclusion

We present a service-aware energy agent framework spanning pricing regimes, failure injection, and verified LLM code-agent repair. The multi-agent controller achieves consistent cost/peak reduction across all scenarios and pricing regimes with a measurable EV service trade-off. Aggressive failure injection confirms correct violation detection (zero resolution on structural violations is correct behavior). The LLM ablations show null divergence between verified/unverified arms

on the safety gate in both fallback mode and the real-LLM M2a screen, isolating the verification machinery as a no-op under an in-bounds proposer; the unverified arm alone shows cost/peak regressions that verification does not target. All artifacts are reproducible: 175 runs on a clean commit with full manifests, zero failures, and seed-pair checks passing.

Future work: longer-horizon M2b confirmation with a harsher LLM proposer, LMP RL policies, P2P market clearing, and extended horizon stress tests.

References

Sture Holm. A simple sequentially rejective multiple test procedure. *Scandinavian Journal of Statistics*, 6(2):65–70, 1979.

NVIDIA. NVIDIA NIM: Microservices for optimized inference on GPUs. *NVIDIA Technical Documentation*, 2024. URL https://www.nvidia.com/en-us/ai-data-science/products/nim-microservices/.

Maik Rall, Paul Boock, Christoph Heinze, and Bernhard Kuchen. Program repair using abstract syntax trees and machine learning. In *IEEE International Conference on Software Maintenance and Evolution (ICSME)*, 2020.

James B. Rawlings, David Q. Mayne, and Moritz Diehl. Model predictive control: Theory, computation, and design. *Nob Hill Publishing*, 2020.

Leon Thurner, Alexander Scheidler, Florian Schäfer, Jan-Hendrik Menke, Julian Dollichon, Frieder Meier, Steffen Meinecke, and Martin Braun. pandapower — an open-source python tool for convenient modeling, analysis, and optimization of electric power systems. In *IEEE Transactions on Power Systems*, volume 33, pages 6510–6521, 2018. doi: 10.1109/TPWRS.2018.2829021.

José R. Vázquez-Canteli, Jérôme Kämpf, Gregor Henze, and Zoltán Nagy. Citylearn v1.0: An openai gym environment for demand response with deep reinforcement learning. *Proceedings of the 6th ACM International Conference on Systems for Energy-Efficient Built Areas (BuildSys)*, pages 176–179, 2020. doi: 10.1145/3408308.3427976.

Xinyang Zhou, Shengjun Gao, and Yu Chen. Multi-agent reinforcement learning for energy management in renewable energy communities. In *IEEE Transactions on Smart Grid*, volume 12, pages 3384–3396, 2021.

Zhi Zhou, Fang Zhao, Jie Wang, and Yan Liu. Deep reinforcement learning for demand response in microgrids: A review. In *ACM e-Energy*, 2022.